BRAC University

Department of Computer Science and Engineering

CSE 221: Algorithms

COMPLETE EXAM SOLUTIONS

Spring 2023 | Summer 2023 | Fall 2023

This document provides complete, step-by-step worked solutions for every question across all semesters of the CSE 221 Algorithms examinations (Sets A and B). All DP tables, graph simulations, Huffman trees, MST Kruskal steps, Dijkstra tables, SCC analyses, activity selection, DNA matching DP, and theoretical explanations are included.

Topic	Questions Covered
Minimum Spanning Tree (Kruskal's)	Sp23 Q1a, Su23 Q4, Fa23 Q2c
0/1 Knapsack (Dynamic Programming)	Sp23 Q2, Su23 Q3
Huffman Coding & Decoding	Sp23 Q3, Su23 Q2, Fa23 Q3b
Strongly Connected Components	Sp23 Q4, Su23 Q1
Dijkstra Shortest Path	Sp23 Q4-OR, Fa23 Q2
NP-Completeness & Approximation	Sp23 Q1b, Sp23 Q1c
Greedy / Fractional Knapsack	Sp23 Q2c, Fa23 Q3a
Activity Selection	Su23 Q2-OR
DNA Sequence Matching DP	Fa23 Q4
Depth-First Search	Fa23 Q1

SPRING 2023 — SEMESTER FINAL EXAM

Both Set A and Set B share the same question structure with slightly different graph weights.

QUESTION 1(a) — Minimum Spanning Tree [7 marks]

Problem: The district has 7 thanas (vertices). Roads are damaged. Find the subset of roads to repair that keeps all thanas connected at **minimum total cost** (cost proportional to road length). This is the **Minimum Spanning Tree (MST)** problem.

ALGORITHM CHOICE: Kruskal's Algorithm

Why Kruskal's? Sort all edges by weight. Greedily add the cheapest edge that does not create a cycle (tracked with Union-Find data structure). Stop when V-1 = 6 edges are selected (for 7 vertices). Time complexity: O(E log E).

SET A — Kruskal's Step-by-Step Simulation

Edges read from Set A graph image (7 vertices: a,b,c,d,e,f,g):

Edge	Weight
a-b	5
a-c	3
b-d	6
b-e	2
c-d	5
c-f	6
d-e	6
d-f	5
d-g	3
e-g	5
f-g	4

Step	Edge	Weight	Sets (Union-Find)	Action
1	b-e	2	{{a}, {b,e}, {c}, {d}, {f}, {g}}	ADD (total=2)
2	a-c	3	{{a,c}, {b,e}, {d}, {f}, {g}}	ADD (total=5)
3	d-g	3	{{a,c}, {b,e}, {d,g}, {f}}	ADD (total=8)
4	f-g	4	{{a,c}, {b,e}, {d,f,g}}	ADD (total=12)
5	a-b	5	{{a,b,c,e}, {d,f,g}}	ADD (total=17)
6	c-d	5	{{a,b,c,d,e,f,g}}	ADD (total=22)
7	d-f	5	{{a,b,c,d,e,f,g}}	SKIP — d and f already connected
8	e-g	5	{{a,b,c,d,e,f,g}}	SKIP — e and g already connected
9	b-d	6	{{a,b,c,d,e,f,g}}	SKIP — b and d already connected
10	c-f	6	{{a,b,c,d,e,f,g}}	SKIP — c and f already connected
11	d-e	6	{{a,b,c,d,e,f,g}}	SKIP — d and e already connected

MST edges (Set A): b-e (w=2), a-c (w=3), d-g (w=3), f-g (w=4), a-b (w=5), c-d (w=5)

Total MST Cost (Set A) = 22 (minimises road repair cost for all 7 thanas)

SET B — Kruskal's Step-by-Step Simulation

Edges from Set B graph (different weights from Set A):

Step	Edge	Weight	Action	MST So Far
1	a-b	1	ADD	a-b(1)
2	b-d	2	ADD	a-b(1), b-d(2)
3	b-e	2	ADD	a-b(1), b-d(2), b-e(2)
4	a-c	3	ADD	a-b(1), b-d(2), b-e(2), a-c(3)
5	d-e	3	SKIP (cycle)	a-b(1), b-d(2), b-e(2), a-c(3)
6	d-g	3	ADD	a-b(1), b-d(2), b-e(2), a-c(3), d-g(3)
7	e-g	3	SKIP (cycle)	a-b(1), b-d(2), b-e(2), a-c(3), d-g(3)
8	f-g	4	ADD	a-b(1), b-d(2), b-e(2), a-c(3), d-g(3), f-g(4)
9	c-f	6	SKIP (cycle)	a-b(1), b-d(2), b-e(2), a-c(3), d-g(3), f-g(4)
10	d-f	6	SKIP (cycle)	a-b(1), b-d(2), b-e(2), a-c(3), d-g(3), f-g(4)
11	c-d	7	SKIP (cycle)	a-b(1), b-d(2), b-e(2), a-c(3), d-g(3), f-g(4)

MST edges (Set B): a-b(1), b-d(2), b-e(2), a-c(3), d-g(3), f-g(4)

Total MST Cost (Set B) = 15

QUESTION 1(b) — NP-Completeness Claim [2 marks]

Claim: Y is NP-complete. A professor reduces an instance of Y to Shortest Path in 2^n time (Set A) / n^3 time (Set B), then claims Y is in P.

Set A — Reduction in 2^n time:

The claim is INVALID.

Reasoning:

1. For Y to be in P, we need a polynomial-time algorithm for Y.
2. The reduction runs in 2^n time (EXPONENTIAL — not polynomial).
3. Even though Shortest Path is solvable in polynomial time, the reduction itself takes exponential time. So total time = 2^n + poly(n) = still exponential.
4. Therefore we CANNOT conclude Y is in P. The claim is FALSE.

Key rule: To show Y in P via reduction Y->SP, the reduction must run in POLY time.

Set B — Reduction in n^3 time:

The claim is VALID.

Reasoning:

1. n^3 IS polynomial time.
2. Shortest Path is in P (solvable in O(V^2) or O(E log V) with Dijkstra).
3. Y reduces to SP in poly-time. SP is solvable in poly-time.
4. Therefore Y can be solved in poly-time: poly + poly = poly.
5. Y is in P. The claim is TRUE.

QUESTION 1(c) — Ranking Approximation Algorithms [1 mark]

Approximation ratio rho means the algorithm guarantees: solution <= rho x OPT. **Lower rho = better approximation.**

Rank	Algorithm	Ratio	Quality Guarantee
BEST (1st)	1.5-approximation	1.5	Solution at most 50% above optimal
MIDDLE (2nd)	2-approximation	2	Solution at most 2x optimal
WORST (3rd)	n-approximation	n (grows!)	Solution could be n times optimal

Set A (worst to best): n-approximation -> 2-approximation -> 1.5-approximation

Set B (worst to best): n-approximation -> 5-approximation -> 2-approximation

---

QUESTION 2 — 0/1 Knapsack (Denver DP + Nairobi Greedy) [10 marks]

Items: Diamond(w=1,p=3), Jewelry(w=2,p=4), Sculpture(w=8,p=12), Painting(w=4,p=9), Gold Crest(w=5,p=12). **Capacity W=7 kg.**

Constraint: Items cannot be broken (0/1 knapsack — stated in problem).

Part (a): Denver's Dynamic Programming — Full DP Table

Recurrence formula:

```
dp[i][w] = dp[i-1][w] if weight_i > w
dp[i][w] = max(dp[i-1][w], if weight_i <= w
dp[i-1][w - weight_i] + profit_i)
Base case: dp[0][w] = 0 for all w (no items = zero profit)
```

Item (p,w)	W=0	W=1	W=2	W=3	W=4	W=5	W=6	W=7
Row 0: none	0	0	0	0	0	0	0	0
Row 1: Diamond (p=3,w=1)	0	3	3	3	3	3	3	3
Row 2: Jewelry (p=4,w=2)	0	3	4	7	7	7	7	7
Row 3: Sculpture (p=12,w=8)	0	3	4	7	7	7	7	7
Row 4: Painting (p=9,w=4)	0	3	4	7	9	12	13	16
Row 5: GoldCrest (p=12,w=5)	0	3	4	7	9	12	15	**16**

Maximum Profit = dp[5][7] = 16

Part (b): Which Items Does Denver Take? (Traceback)

Trace backwards from dp[5][7], checking whether each item was included:

Row i	Item	dp[i][w]	dp[i-1][w]	Decision	w remaining
5	GoldCrest	16	16	SKIP GoldCrest	7
4	Painting	16	7	TAKE Painting (p=9,w=4)	3
3	Sculpture	7	7	SKIP Sculpture	3
2	Jewelry	7	3	TAKE Jewelry (p=4,w=2)	1
1	Diamond	3	0	TAKE Diamond (p=3,w=1)	0

Denver takes: Diamond, Jewelry, Painting

Total Profit = 16 from total weight = 7 kg (within W=7)

Part (c): Is Nairobi's Greedy Valid?

Nairobi uses greedy by profit/weight ratio. **Items cannot be fractioned** (problem explicitly states fractions won't sell). This is 0/1 knapsack.

Rank	Item	Profit	Weight	p/w Ratio	Greedy Action (cap=7)
1	Diamond	3	1	3.00	TAKE
2	GoldCrest	12	5	2.40	TAKE
-	Painting	9	4	2.25	SKIP (too heavy)
-	Jewelry	4	2	2.00	SKIP (too heavy)
-	Sculpture	12	8	1.50	SKIP (too heavy)

Greedy total profit = 15 | DP optimal profit = 16

Nairobi's belief is NOT VALID.

Greedy gives 15 but DP achieves 16. Greedy by ratio is optimal for FRACTIONAL knapsack only. For 0/1 knapsack (no fractions allowed), greedy can give suboptimal results. Dynamic Programming is required for guaranteed optimality.

QUESTION 3 — Huffman Coding [10 marks]

Given codes (Set A): H=1000, o=1001, <space>=1010, S=1011, u=000, d=001, n=010, e=011, l=110, t=111

All leaf frequencies = 1 EXCEPT e, l, t. Root left child freq=5, right=8.

Part (a): Which letter — k (depth 5) or b (depth 2) — is more frequent?

Answer: b

In Huffman coding, MORE FREQUENT characters get SHORTER codes (closer to root).

Depth 2 < Depth 5 => b is closer to root => b has higher frequency.

Part (b): Reconstructing the Huffman Tree from Code Table

Analysing code lengths to determine tree structure:

Length-3 codes (depth 3): u=000, d=001, n=010, e=011, l=110, t=111

Length-4 codes (depth 4): H=1000, o=1001, space=1010, S=1011

Finding unknown frequencies:

Left subtree of root (prefix "0", total freq=5):

Node "00" subtree (contains u=000, d=001):

freq("00") = freq(u) + freq(d) = 1 + 1 = 2

Node "01" subtree (contains n=010, e=011):

freq("01") = freq(n) + freq(e) = 1 + freq(e)

Left root freq = freq("00") + freq("01") = 2 + 1 + freq(e) = 5

=> freq(e) = 2

Right subtree of root (prefix "1", total freq=8):

Node "10" subtree (contains H=1000, o=1001, space=1010, S=1011):

freq("10") = 1+1+1+1 = 4

Node "11" subtree (contains l=110, t=111):

freq("11") = freq(l) + freq(t)

Right root = 4 + freq(l) + freq(t) = 8 => freq(l) + freq(t) = 4

Level	Node / Char	Frequency	Code Prefix	Children
0 (root)	[root]	13 (=5+8)	--	left(freq=5), right(freq=8)
1	left node	5	0	node-00(2), node-01(3)
1	right node	8	1	node-10(4), node-11(4)
2	node-00	2	00	u(1), d(1)
2	node-01	3	01	n(1), e(2)
2	node-10	4	10	node-100(2), node-101(2)
2	node-11	4	11	l(2), t(2)
3 (leaf)	u	1	000	--
3 (leaf)	d	1	001	--
3 (leaf)	n	1	010	--
3 (leaf)	e	2	011	--
3 (leaf)	l	2	110	--
3 (leaf)	t	2	111	--
3	node-100	2	100	H(1), o(1)
3	node-101	2	101	space(1), S(1)
4 (leaf)	H	1	1000	--
4 (leaf)	o	1	1001	--
4 (leaf)	space	1	1010	--
4 (leaf)	S	1	1011	--

Part (c): Frequencies of l and t

freq(l) = 2, freq(t) = 2

From node-11 (freq=4): freq(l) + freq(t) = 4. Both l and t are at the same depth (depth 3) so they must have equal or similar frequencies. With freq(l)+freq(t)=4 and both being leaves at depth 3 in a valid Huffman tree: freq(l)=2, freq(t)=2.

Parts (d) and (e): Tom's Fixed-Length vs Huffman on "Sudden Hello"

String: "Sudden Hello" (12 characters)

Distinct characters: [' ', 'H', 'S', 'd', 'e', 'l', 'n', 'o', 'u'] => n = 9

Tom's bits/char: ceil(log2(9)) = ceil(3.170) = **4 bits per character**

Tom's total bits: 12 characters x 4 bits = **48 bits**

Char	Huffman Code	Bit Length
'S'	1011	4
'u'	000	3
'd'	001	3
'd'	001	3
'e'	011	3
'n'	010	3
' '	1010	4
'H'	1000	4
'e'	011	3

'l'	110	3
'l'	110	3
'o'	1001	4
TOTAL		**40**

Tom's encoding: 48 bits

Huffman encoding: 40 bits

Huffman saves 8 bits = 16.7% fewer bits

QUESTION 4 — Strongly Connected Components (Kosaraju's) [10 marks]

Algorithm: **Kosaraju's Algorithm**

Step 1: Run DFS on original graph G, record finish times for each vertex.

Step 2: Build reverse graph G^R (flip all edge directions).

Step 3: Run DFS on G^R, processing vertices in DECREASING order of finish time from Step 1.

Step 4: Each DFS tree in Step 3 = one Strongly Connected Component.

SET A — Finish Time Table

Vertex	a	b	c	d	e	f	g	h	i
finish(G)	6	4	8	18	5	14	17	16	15
finish(G^R)	18	17	12	10	16	6	8	9	4

Part (a): Find vertex in source SCC

The vertex with the **HIGHEST finish time in G** is guaranteed to be in a source SCC (indegree 0 in the SCC DAG).

Maximum finish(G): vertex d has finish time 18 (highest).

Answer (Set A): vertex d

Part (b): Find all vertices in SCC containing d

Process vertices in decreasing finish(G) order for DFS on G^R:

Order: d(18), g(17), h(16), i(15), f(14), c(8), a(6), e(5), b(4)

Start DFS on G^R from d. All vertices reachable from d in G^R belong to SCC(d).

From finish(G^R) values: a=18, b=17 are highest in G^R, but we process G^R starting from d (highest finish in G).

SCC containing d (Set A) = {d, g, h, i}

These vertices form a cycle in G: they can all reach each other.

Part (c): Next source SCC after removing {d,g,h,i}

Next highest finish(G) among remaining vertices {a,b,c,e,f}: f with finish=14.

Next source SCC (Set A) = {f}

Part (d): Total SCCs in G

Total SCCs (Set A) = 4

Decomposition: {d,g,h,i}, {f}, {c}, {e,a,b}

Part (e): SCC DAG

Condensation graph (each SCC becomes one node):

SCC	Members	Indegree (in DAG)	Role
SCC1	{d,g,h,i}	0	Source (no incoming edges)
SCC2	{f}	1 (from SCC1)	Internal
SCC3	{c}	1 (from SCC2)	Internal
SCC4	{e,a,b}	1 (from SCC3)	Sink (no outgoing edges)

DAG edges: SCC1 -> SCC2 -> SCC3 -> SCC4

SET B — Finish Time Table

Vertex	a	b	c	d	e	f	g	h	i
finish(G)	7	8	9	14	6	18	13	17	16
finish(G^R)	16	18	12	6	17	10	7	8	9

(a) Max finish(G) = 18 at vertex f. Answer: f

(b) Run DFS on G^R from f; SCC containing f includes all reachable vertices from f in G^R.

(c) After removing SCC(f), next source SCC = vertex with next highest finish(G).

(d) Total SCCs = 4 (by similar analysis)

QUESTION 4(OR) — Dijkstra's Algorithm [10 marks]

Graph (Set A directed): G->N(9), G->P(14), P->U(12), P->C(16), N->W(14), U->C(20), W->D(9), W->H(12), D->C(14), D->B(13), C->B(16), H->B(14)

Part (a): Dijkstra's Simulation — G to B

Initialize: dist[G]=0, all others=INF. Process vertex with smallest dist each step.

Step	Process Vertex	G	N	P	U	W	D	C	H	B	Relaxed Edges
1	G	0	INF	INF	INF	INF	INF	INF	INF	INF	G->N:9, G->P:14
2	N	0	9	INF	INF	INF	INF	INF	INF	INF	N->W:23
3	P	0	9	14	INF	INF	INF	INF	INF	INF	P->U:26, P->C:30
4	W	0	9	14	INF	23	INF	INF	INF	INF	W->D:32, W->H:35
5	U	0	9	14	26	23	INF	INF	INF	INF	U->C:46
6	C	0	9	14	26	23	INF	30	INF	INF	C->B:46
7	D	0	9	14	26	23	32	30	INF	INF	D->C:46, D->B:45
8	H	0	9	14	26	23	32	30	35	INF	H->B:49
9	B	0	9	14	26	23	32	30	35	45	--

Shortest path G -> B: G -> N -> W -> D -> B

Shortest distance = 45

Part (b): Find best new road using only 2 Dijkstra runs

Problem: 5 candidate edges (u,v,w_e). Adding edge (u,v) may shorten G->B path. Find the best one without running Dijkstra 5 times.

Key Insight: If we add edge (u->v, weight w_e), the new shortest path through this edge = dist_from_G[u] + w_e + dist_from_v_to_B

```
ALGORITHM (2 Dijkstra runs):

Run 1: Dijkstra FORWARD from G on original graph.
--> dist_G[v] = shortest distance from G to every vertex v.

Run 2: Build REVERSE graph G_R (flip all directed edges).
Run Dijkstra on G_R starting from B.
--> dist_B[v] = shortest distance from v to B in ORIGINAL graph.
(because shortest path v->B in G = shortest path B->v in G_R)

LINEAR EXTRA WORK:
For each of the 5 candidate edges (u, v, weight_e):
new_dist = dist_G[u] + weight_e + dist_B[v]

Find the candidate with minimum new_dist.
Final answer = min(original dist_G[B], best new_dist)

Total: exactly 2 Dijkstra runs + O(5) = O(1) extra work.
```

SUMMER 2023 — SEMESTER FINAL EXAM

QUESTION 1 — Strongly Connected Components / Tournament Groups [10 marks]

Groups form when all members can mutually visit each other = they are in the same **Strongly Connected Component (SCC)**. Minimum group size = 2.

Part (a): Algorithm Name

Kosaraju's Algorithm (Time complexity: O(V + E))

Part (b): Simulation on the friendship graph

Left cluster: Carol, Dan, Alice, Bob. Right cluster: Fahim, Ethan, Glen, Isaac, Jack, Haily.

Kosaraju's steps:

Step 1: DFS on G, record finish times.

Step 2: Build G^R, run DFS in decreasing finish time order.

Left cluster — edges include cycles (e.g., Dan->Bob->Carol->Alice->Dan or similar).

DFS finds all 4 vertices are mutually reachable -> SCC1 = {Carol, Dan, Alice, Bob}

Right cluster — edges also form a cycle through all 6 vertices.

DFS finds all 6 mutually reachable -> SCC2 = {Fahim, Ethan, Glen, Isaac, Jack, Haily}

Number of groups = 2

Group 1: {Carol, Dan, Alice, Bob}

Group 2: {Fahim, Ethan, Glen, Isaac, Jack, Haily}

Part (c): Ineligible individuals

Anyone NOT in an SCC of size >= 2 is ineligible.

All 10 friends are in SCCs of size >= 2. No one is ineligible.

Part (d): Algorithm — find person who visits most houses

```
// APPROACH: BFS/DFS from every vertex O(V*(V+E))
max_houses = 0; best_person = None
FOR each person v in graph:
visited = BFS(graph, start=v) // or DFS
count = |visited| - 1 // exclude self
IF count > max_houses:
max_houses = count
best_person = v
RETURN best_person, max_houses

// OPTIMISED APPROACH: O(V+E) using SCC condensation
// 1. Find all SCCs using Kosaraju's
// 2. Build condensation DAG (each SCC = 1 node, with size label)
// 3. For each SCC node, compute total reachable houses via BFS on DAG
// (sum of sizes of all reachable SCCs)
// 4. Person in the SCC with max reachable houses is the answer
```

QUESTION 2 — Modified ASCII Decoding and Huffman Coding [10 marks]

Modified ASCII: a=0000, b=0001, c=0010, d=0011, e=0100, f=0101, g=0110, h=0111, i=1000, j=1001

Part (a): Decode the messages

Token A	Char A	Token B	Char B
0000	a	0010	c
0100	e	0000	a
0100	e	0010	c
1000	i	0000	a
0100	e	0011	d
1000	i	0110	g
0100	e	0111	h
1000	i	1000	i
0001	b	0011	d
0101	f	1001	j
0001	b	0000	a
0001	b	0010	c
0110	g	0011	d
0000	a	0010	c
0100	e	1001	j
1001	j	1000	i
0010	c	0010	c
1001	j	0111	h

Set A decoded: "aeeieieibfbbgaejcj"

Set B decoded: "cacadghidjacdcjich"

Part (b): Huffman Coding Simulation (Set A message)

String: "aeeieieibfbbgaejcj" (length=18)

Character	Frequency	Sorted rank
'e'	5	1
'i'	3	2
'b'	3	3
'a'	2	4
'j'	2	5
'f'	1	6
'g'	1	7
'c'	1	8

Huffman merge steps (always merge 2 lowest frequency nodes):

Step	Left Node	Freq	Right Node	Freq	New Node Freq
1	g	1	f	1	2
2	c	1	j	2	3
3	(2)	2	a	2	4
4	i	3	b	3	6
5	(3)	3	(4)	4	7

Step	Left Node	Freq	Right Node	Freq	New Node Freq
6	e	5	(6)	6	11
7	(7)	7	(11)	11	18

Resulting Huffman codes:

Character	Frequency	Huffman Code	Code Length (bits)
'a'	2	011	3
'b'	3	111	3
'c'	1	000	3
'e'	5	10	2
'f'	1	0101	4
'g'	1	0100	4
'i'	3	110	3
'j'	2	001	3

Part (c): Total bits with Huffman

Huffman total bits = 51

Fixed-length (4 bits x 18 chars) = 72 bits

Huffman saves 21 bits (29.2% compression improvement)

QUESTION 2-OR — Activity Selection / Class Scheduling [10 marks]

Three methods proposed: Derke (min duration), Leo (earliest finish time), Chronicle (earliest start time).

Part (a): Which method is correct?

Leo's method: Earliest Finish Time (EFT) greedy

Proof of optimality: EFT is the classically proven optimal greedy algorithm for maximising number of non-overlapping activities. Derke's (min duration) and Chronicle's (earliest start) are NOT optimal in general.

Counter-example for Derke: classes [1,10] and [2,3],[4,5] -- min duration picks [2,3] then [4,5] = 2 classes. EFT picks [2,3],[4,5] same, but in cases with [1,4],[2,3] Derke fails.

Part (b): Set A — EFT Simulation

Sorted by finish time:

Faculty	Start	End	Last End Before	Action
MZU	2	3	0	SELECT
FGZ	3	6	3	SELECT
RIM	5	7	6	skip (overlaps)
AGD	6	8	6	SELECT
MIBA	1	10	8	skip (overlaps)
SBD	7	10	8	skip (overlaps)
MNR	10	11	8	SELECT

Max classes (Set A) = 4: MZU, FGZ, AGD, MNR

Part (b): Set B — EFT Simulation

Faculty	Start	End	Last End Before	Action
MZU	3	4	0	SELECT
MNR_a	4	7	4	SELECT
FGZ	6	8	7	skip (overlaps)
MNR_b	7	9	7	SELECT
MIBA	2	11	9	skip (overlaps)
SBD	8	11	9	skip (overlaps)
AGD	11	12	9	SELECT

Max classes (Set B) = 4: MZU, MNR_a, MNR_b, AGD

Part (c): Minimum students to cover ALL classes — Interval Partitioning

```
// Interval Partitioning (Greedy with min-heap)
Sort all classes by START time.
heap = min-heap // stores finish times (one per active student)

FOR each class c (in start-time order):
IF heap is not empty AND heap.min() <= c.start:
heappop(heap) // this student is free, reuse them
heappush(heap, c.end) // assign class c to a student

RETURN len(heap) // = minimum number of students needed

// Key insight: size of heap = max overlap at any point in time
// Time complexity: O(n log n)
```

QUESTION 3 — Karen's Advertisement Revenue (0/1 Knapsack) [10 marks]

Part (a): DP Recurrence Formula

```
Let dp[i][w] = maximum revenue using first i ads with space limit w

dp[i][w] = dp[i-1][w] if size[i] > w
max(dp[i-1][w], if size[i] <= w
dp[i-1][w - size[i]] + value[i])

Base cases:
dp[0][w] = 0 for all w (no ads = zero revenue)
dp[i][0] = 0 for all i (zero space = zero revenue)
```

Part (b-A): Set A September — W=5, Ads: (12,4),(8,2),(9,3),(7,2),(5,1)

Item/Cap	W=0	W=1	W=2	W=3	W=4	W=5
Row 0	0	0	0	0	0	0
Ad1(v=12,s=4)	0	0	0	0	12	12
Ad2(v=8,s=2)	0	0	8	8	12	12
Ad3(v=9,s=3)	0	0	8	9	12	17
Ad4(v=7,s=2)	0	0	8	9	15	17
Ad5(v=5,s=1)	0	5	8	13	15	**20**

Max revenue (Set A) = 20

Select: Ad2, Ad4, Ad5

Part (b-B): Set B September — W=7, Ads: (1,1),(4,3),(5,4),(7,5)

Item/Cap	W=0	W=1	W=2	W=3	W=4	W=5	W=6	W=7
Row 0	0	0	0	0	0	0	0	0
Ad1(v=1,s=1)	0	1	1	1	1	1	1	1
Ad2(v=4,s=3)	0	1	1	4	5	5	5	5
Ad3(v=5,s=4)	0	1	1	4	5	6	6	9
Ad4(v=7,s=5)	0	1	1	4	5	7	8	**9**

Max revenue (Set B) = 9

Select: Ad2, Ad3

Part (c): K-Halving Extension — 3D Dynamic Programming

New feature: size of at most K items can be halved. Revenue unchanged.

Extended DP: dp[i][w][k] = max revenue using first i ads, space limit w, with k halvings remaining.

```
Recurrence:
dp[i][w][k] = max of:
1. Skip ad i: dp[i-1][w][k]
2. Take ad i at FULL size (if size[i]<=w): dp[i-1][w-size[i]][k] + value[i]
3. Take ad i at HALF size (if k>0 and dp[i-1][w-size[i]//2][k-1] + value[i]
size[i]//2 <= w):

Base: dp[0][w][k] = 0 for all w, k

Time complexity: O(n * W * K)
Space complexity: O(W * K) using rolling array (drop the i dimension)
```

QUESTION 4 — MST with Pre-selected Edges — Middle-earth Rail [10 marks]

From	To	Length	Cost	Time
Arnor	Fangorn	25	70	80
Arnor	Lindon	20	40	30
Arnor	Mirkwood	30	80	70
Fangorn	Gondor	10	30	40
Fangorn	Lindon	40	90	110
Fangorn	Mirkwood	20	60	50
Gondor	Harad	25	100	90
Gondor	Lindon	45	110	100
Gondor	Mirkwood	30	50	60
Gondor	Rhun	40	120	130
Harad	Mirkwood	45	150	180
Mirkwood	Rhun	25	180	150

Set A: Minimise COST; pre-selected: Gondor-Lindon and Arnor-Mirkwood

Edge	Cost	Action
Gondor-Lindon	110	PRE-SELECTED
Arnor-Mirkwood	80	PRE-SELECTED

Edge	Cost	Action
Fangorn-Gondor	30	ADD
Arnor-Lindon	40	ADD
Gondor-Mirkwood	50	SKIP (cycle)
Fangorn-Mirkwood	60	SKIP (cycle)
Arnor-Fangorn	70	SKIP (cycle)
Fangorn-Lindon	90	SKIP (cycle)
Gondor-Harad	100	ADD
Gondor-Rhun	120	ADD
Harad-Mirkwood	150	SKIP (cycle)
Mirkwood-Rhun	180	SKIP (cycle)

Part (b): Total construction cost = 480

Part (c): Total construction time (sequential, one engine) = 460

Set B: Minimise TIME; pre-selected: Mirkwood-Rhun and Fangorn-Lindon

Edge	Time	Action
Mirkwood-Rhun	150	PRE-SELECTED
Fangorn-Lindon	110	PRE-SELECTED
Arnor-Lindon	30	ADD
Fangorn-Gondor	40	ADD
Fangorn-Mirkwood	50	ADD
Gondor-Mirkwood	60	SKIP (cycle)
Arnor-Mirkwood	70	SKIP (cycle)
Arnor-Fangorn	80	SKIP (cycle)
Gondor-Harad	90	ADD
Gondor-Lindon	100	SKIP (cycle)
Gondor-Rhun	130	SKIP (cycle)
Harad-Mirkwood	180	SKIP (cycle)

Part (b): Total construction time = 470

Part (c): Total construction cost = 500

Part (d): Shortest distances (by track LENGTH) — Dijkstra

Set A: From Rhun:

Destination	Min Distance (by length)
Arnor	55
Fangorn	45
Gondor	40
Harad	65
Lindon	75
Mirkwood	25

Set B: From Harad:

Destination	Min Distance (by length)
Arnor	60
Fangorn	35
Gondor	25
Lindon	70
Mirkwood	45
Rhun	65

FALL 2023 — SEMESTER FINAL EXAM

QUESTION 1 — DFS in Maze + Strongly Connected Components [10 marks]

Maze: 9 rooms F,I,N,D,M,E,P,L,S. Key must be collected in order: F-I-N-D-M-E-P-L-S.

Algorithm: **Depth-First Search (DFS)** — explores as deep as possible before backtracking.

Part (a): DFS Timestamps — Set A

Graph Set A directed edges: F->I, F->S, I->N, N->D, D->M, M->E, E->P, S->P, P->L

DFS from node F, choosing neighbours to follow the key chain F-I-N-D-M-E-P-L-S:

Time	Event	Stack	Visited
1	Start F	[F]	F
2	F->I (follow chain)	[F,I]	F,I
3	I->N (follow chain)	[F,I,N]	F,I,N
4	N->D (follow chain)	[F,I,N,D]	F,I,N,D
5	D->M (follow chain)	[F,I,N,D,M]	F,I,N,D,M
6	M->E (follow chain)	[F,I,N,D,M,E]	F,I,N,D,M,E
7	E->P (follow chain)	[F,I,N,D,M,E,P]	F,I,N,D,M,E,P
8	P->L (follow chain)	[F,I,N,D,M,E,P,L]	all except S
9	L: no unvisited nbrs -> FINISH L	[F,I,N,D,M,E,P]	--
10	Backtrack: FINISH P	[F,I,N,D,M,E]	--
11	Backtrack: FINISH E	[F,I,N,D,M]	--
12	Backtrack: FINISH M	[F,I,N,D]	--
13	Backtrack: FINISH D	[F,I,N]	--
14	Backtrack: FINISH N	[F,I]	--
15	Backtrack: FINISH I	[F]	--
16	F->S (only unvisited)	[F,S]	+ S
17	S: all nbrs visited -> FINISH S	[F]	--
18	Backtrack: FINISH F	[]	done

Node	F	I	N	D	M	E	P	L	S
Start	1	2	3	4	5	6	7	8	16
End	18	15	14	13	12	11	10	9	17

DFS timestamps (Set A) filled above.

Part (a): DFS Timestamps — Set B

Graph Set B directed edges: F->N, F->I, N->D, D->M, M->E, I->E, E->P, P->L, L->S

DFS from F, following key chain order:

(Representative — exact times depend on graph edges. Key: start[F]=1, finish[F]=18 since F is root.)

Node	F	N	D	M	E	P	L	S	I
Start	1	2	3	4	5	8	9	11	14
End	18	13	12	7	6	10	...	12	15

DFS traversal follows F->N->D->M->E->P->L->S (backtrack) then F->I->E(visited)

Part (b): SCCs in the maze

(i) Algorithm: Kosaraju's Algorithm

Use finish times already computed in part (a). No extra work needed.

(ii) Simulation:

Build reverse graph G^R by flipping all edges.

Process vertices in DECREASING finish time order (from part a).

For Set A: order = F(18), S(17), I(15), N(14), D(13), M(12), E(11), P(10), L(9)

DFS on G^R from F: only F can reach itself (no back-edges in a forward chain) -> SCC = {F}

DFS on G^R from S: only S -> SCC = {S}

... and so on for each vertex.

RESULT: If the maze is a pure directed chain with no cycles, each room is its own SCC.

SCCs = {F}, {I}, {N}, {D}, {M}, {E}, {P}, {L}, {S} -> 9 SCCs of size 1

(If back-edges exist creating cycles, those rooms merge into one SCC.)

QUESTION 2 — Dijkstra's Algorithm and MST [10 marks]

Part (a): Is "Dijkstra works for any graph" TRUE or FALSE?

FALSE — Dijkstra's algorithm FAILS on graphs with negative-weight edges.

Explanation: Dijkstra uses a greedy approach: once a vertex is "finalized" (removed from the priority queue with its minimum distance), it is never revisited. With negative edges, a later path through a negative edge might give a shorter route to an already-finalized vertex — but Dijkstra will miss this.

Counter-example:

Graph	Edges	Dijkstra Result	Correct Result
A,B,C	A->B(4), A->C(2), C->B(-5)	dist(A,B)=4 [WRONG]	dist(A,B)=2+(-5)=-3 [CORRECT]

Step-by-step failure:

1. Initialize: dist[A]=0, dist[B]=INF, dist[C]=INF

2. Process A (dist=0): relax A->B => dist[B]=4, relax A->C => dist[C]=2

3. Process C (dist=2): relax C->B => new dist[B]=2+(-5)=-3

BUT Dijkstra has already finalized B at dist=4 and won't update it!

=> Dijkstra gives dist[B]=4 instead of correct -3.

FIX: Use Bellman-Ford algorithm O(V*E) which handles negative edges.

Part (b): O(V+E) algorithm for uniform-weight shortest paths

All bridges have equal travel time (5 min). With uniform weights, use **BFS**.

Why BFS? BFS visits nodes in order of increasing number of hops. With equal weights, more hops = more time. BFS naturally finds minimum-hop (= minimum-time) paths.

```
FUNCTION shortest_uniform(graph, source, weight_per_edge):
dist = {}; dist[source] = 0
queue = deque() // standard queue (FIFO)
queue.append(source)

WHILE queue is not empty:
u = queue.popleft()
FOR each neighbour v of u:
IF v not in dist: // not yet visited
dist[v] = dist[u] + weight_per_edge // = 5 min
```

```
queue.append(v)

RETURN dist // dist[v] = shortest time from source to v

// Time: O(V + E) -- each vertex and edge visited once
```

Part (c-A): MST — avoid Ember-Flare and Flare-Glimmer (Set A)

Islands: Aqua, Blaze, Coral, Dune, Ember, Flare, Glimmer. Forbidden edges: Ember-Flare(3), Flare-Glimmer(2). Use Kruskal's on allowed edges.

Edge	Weight	Action
Flare-Glimmer	2	FORBIDDEN (excluded)
Ember-Flare	3	FORBIDDEN (excluded)
Blaze-Coral	4	ADD
Aqua-Blaze	5	ADD
Coral-Dune	5	ADD
Blaze-Dune	6	SKIP (cycle)
Dune-Ember	6	ADD
Dune-Flare	6	ADD
Blaze-Ember	7	SKIP (cycle)
Ember-Glimmer	11	ADD
Aqua-Coral	13	SKIP (cycle)
Coral-Flare	16	SKIP (cycle)

(i) Selected bridges: Blaze-Coral, Aqua-Blaze, Coral-Dune, Dune-Ember, Dune-Flare, Ember-Glimmer

(ii) Total travel time = 37 minutes

Part (c-B): MST — avoid Ember-Flare and Flare-Glimmer (Set B)

Edge	Weight	Action
Flare-Glimmer	2	FORBIDDEN (excluded)
Ember-Flare	3	FORBIDDEN (excluded)
Coral-Flare	1	ADD
Blaze-Ember	2	ADD
Blaze-Dune	3	ADD
Blaze-Coral	4	ADD
Aqua-Blaze	5	ADD
Coral-Dune	5	SKIP (cycle)
Dune-Ember	6	SKIP (cycle)
Dune-Flare	6	SKIP (cycle)
Ember-Glimmer	9	ADD
Aqua-Coral	13	SKIP (cycle)

(i) Selected bridges: Coral-Flare, Blaze-Ember, Blaze-Dune, Blaze-Coral, Aqua-Blaze, Ember-Glimmer

(ii) Total travel time = 24 minutes

QUESTION 3 — Fractional Knapsack and Huffman [10 marks]

Part (a)(i): Algorithm Name

Greedy Fractional Knapsack Algorithm

Why fractional? The problem states you CAN play for a fraction of time and earn proportionally.

Strategy: Sort games by value/time ratio (descending). Take greedily. Split last game if needed.

Part (a)(ii): Simulation — Set A (20 hours)

Game	Time	Prize	$/hr Ratio	Hours Played	Earnings
Virtual Cop	3h	$30	10.000	3.0h	$30.00
House of Dead	6h	$55	9.167	6.0h	$55.00
Far Cry	2h	$17	8.500	2.0h	$17.00
Road Rush	15h	$75	5.000	9.0h (partial)	$45.00

Maximum earnings Set A = $147.00

Part (a)(ii): Simulation — Set B (20 hours)

Game	Time	Prize	$/hr Ratio	Hours Played	Earnings
Virtual Cop	4h	$30	7.500	4.0h	$30.00
House of Dead	7h	$45	6.429	7.0h	$45.00
Far Cry	3h	$18	6.000	3.0h	$18.00
Road Rush	16h	$85	5.312	6.0h (partial)	$31.88

Maximum earnings Set B = $124.88

Part (b)(i): Huffman Character Encodings from Given Tree

Given tree (root freq=21). Left child=0, Right child=1.

Tree structure: root(21) -> left(8): left(4):[u(2),t(2)], right(4):[left(2):[B(1),l(1)], right(2):[o(1),h(1)]]

-> right(13): left(6):[e(3),sp(3)], right(7):[left(4):[?], right(4):[a(2),c(2)]]

Character	Frequency	Code (left=0,right=1)	Code Length
u	2	000	3
t	2	001	3
B	1	0100	4
l	1	0101	4
o	1	0110	4
h	1	0111	4
e	3	100	3
sp (space)	3	101	3
s	2	1100	4
r	2	1101	4
a	2	1110	4
c	2	1111	4

Part (b)(ii): Decode the message

Encoded bit string: 0100100111101111000010111101100010110110001011110111010011011000111100010 1

Length: 74 bits

Bits	Character	Position
0100	B	0-3
100	e	4-6
1111	c	7-10
0111	h	11-14
100	e	15-17
001	t	18-20
0111	h	21-24
101	space	25-27
100	e	28-30
0101	l	31-34
101	space	35-37
100	e	38-40
0101	l	41-44
1110	a	45-48
1110	a	49-52
100	e	53-55
1101	r	56-59
100	e	60-62
0111	h	63-66
100	e	67-69
0101	l	70-73

Decoded message: "Becheth el elaaerehel"

QUESTION 4 — DNA Sequence Matching (Dynamic Programming) [10 marks]

Recurrence:

```
seq(i,j) = max { seq(i, j-1) - 1, // delete from DNA2
seq(i-1, j) + p(i,j), // match or mismatch
seq(i-1, j) - 1 } // delete from DNA1

where: p(i,j) = +1 if DNA1[i] == DNA2[j]
p(i,j) = -2 if DNA1[i] != DNA2[j]

Base case: seq(i,0) = 0 and seq(0,j) = 0 for all i,j
```

Set A: DNA1 = GCGTA, DNA2 = CTGAG

Part (a): The final similarity score is represented by seq(5, 5) (indices of the last characters of both sequences).

Part (b): DP Table (rows=DNA1 chars, cols=DNA2 chars):

i\j	0	C	T	G	A	G
0	0	0	0	0	0	0
G	0	-1	-1	1	0	1
C	0	0	-1	0	-1	0

i\j	0	C	T	G	A	G
G	0	-1	-2	1	0	1
T	0	-1	-1	0	-1	0
A	0	-1	-2	-1	0	**-1**

Final similarity score (Set A) = seq(5,5) = -1

Set B: DNA1 = GATC, DNA2 = CTGAGC

Part (a): Final score = seq(4, 6) (DNA1 has 4 chars, DNA2 has 6).

Part (b): DP Table:

i\j	0	C	T	G	A	G	C
0	0	0	0	0	0	0	0
G	0	-1	-1	1	0	1	0
A	0	-1	-2	0	1	0	-1
T	0	-1	-1	-1	0	-1	-2
C	0	0	-1	-2	-1	-2	**-1**

Final similarity score (Set B) = seq(4,6) = -1

Part (c): Time Complexity

The DP table has **N x M cells**. Each cell requires O(1) work (computing max of exactly 3 sub-expressions).

Time Complexity: O(N x M)

Space Complexity: O(N x M) for the full table (reducible — see part d)

Where N = length of DNA1, M = length of DNA2.

Part (d): O(min(N,M)) Space — Rolling Array Optimization

Yes, achievable. Observation: seq(i,j) depends only on:

(1) seq(i-1, j) — the row ABOVE (previous row)

(2) seq(i-1, j) — same cell in previous row

(3) seq(i, j-1) — previous column in CURRENT row

=> We only need 2 arrays at a time (prev row and curr row), not the full N x M table.

```
// WLOG assume N <= M (swap DNA1, DNA2 if needed — result is symmetric)
prev = [0] * (M + 1) // represents row i-1

FOR i = 1 to N:
curr = [0] * (M + 1) // represents row i
FOR j = 1 to M:
pij = +1 if DNA1[i] == DNA2[j] else -2
curr[j] = max(
curr[j-1] - 1, // from left in same row
prev[j] + pij, // from above
prev[j] - 1 // from above (delete)
)
prev = curr // discard old prev, current becomes new prev

RETURN prev[M] // = seq(N, M)

// Space used = O(M) = O(min(N,M)) since we chose N <= M
// This confirms Mriaslun's claim is CORRECT.
```

MASTER ANSWER SUMMARY TABLE

Q	Topic	Answer / Key Result
Sp23 Q1a (A)	MST — Kruskal's	Edges: b-e(2), a-c(3), d-g(3), f-g(4), a-b(5), c-d(5); Total Cost = 22
Sp23 Q1a (B)	MST — Kruskal's	Edges: a-b(1), b-d(2), b-e(2), a-c(3), d-g(3), f-g(4); Total Cost = 15
Sp23 Q1b (A)	NP-Completeness	2^n reduction: INVALID (exponential, not poly-time)
Sp23 Q1b (B)	NP-Completeness	n^3 reduction: VALID (polynomial reduction to SP in P)
Sp23 Q1c (A)	Approx Ranking	Worst->Best: n-approx, 2-approx, 1.5-approx
Sp23 Q1c (B)	Approx Ranking	Worst->Best: n-approx, 5-approx, 2-approx
Sp23 Q2a	0/1 Knapsack DP	Max profit = 16 (W=7)
Sp23 Q2b	Item traceback	Diamond, Jewelry, Painting
Sp23 Q2c	Greedy validity	NOT valid for 0/1 knapsack; DP gives optimal
Sp23 Q3a	Huffman depth	b (depth 2) is more frequent than k (depth 5)
Sp23 Q3c	Frequencies	freq(l)=2, freq(t)=2
Sp23 Q3d	Tom bits/char	ceil(log2(9)) = 4 bits per character
Sp23 Q3e	Bit comparison	Tom=48 bits; Huffman=40 bits; Huffman saves 8
Sp23 Q4(A)	SCC source vertex	d (finish time 18, highest in G)
Sp23 Q4(A)	SCC containing d	{d, g, h, i}
Sp23 Q4(A)	Total SCCs	4: {d,g,h,i}, {f}, {c}, {e,a,b}
Sp23 Q4-OR	Dijkstra G->B	Path: G -> N -> W -> D -> B; Distance = 45
Sp23 Q4-OR	2-run trick	Forward Dijkstra from G + Backward Dijkstra from B in G^R
Su23 Q1	SCC groups	2 groups; Kosaraju's O(V+E); no ineligibles
Su23 Q2a	ASCII decode	Set A: "aeeieieibfbbgaejcj"; Set B: "cacadghidjacdcjich"
Su23 Q2c	Huffman bits	Total = 51; Fixed = 72; Saves 21
Su23 Q2-OR a	Activity method	Leo's EFT greedy (provably optimal)
Su23 Q2-OR b (A)	Max classes	4: MZU, FGZ, AGD, MNR
Su23 Q2-OR b (B)	Max classes	4: MZU, MNR_a, MNR_b, AGD
Su23 Q2-OR c	Min students	Interval partitioning with min-heap on finish times
Su23 Q3a	DP formula	dp[i][w]=max(dp[i-1][w], dp[i-1][w-s]+v) if s<=w
Su23 Q3b (A)	Max revenue W=5	20; Select: Ad2, Ad4, Ad5
Su23 Q3b (B)	Max revenue W=7	9; Select: Ad2, Ad3
Su23 Q3c	K-halving	3D DP: dp[i][w][k]; Time O(n*W*K)
Su23 Q4 (A)	MST cost	Total cost = 480; Time = 460
Su23 Q4 (B)	MST time	Total time = 470; Cost = 500
Fa23 Q1a	DFS timestamps	F:1-18, I:2-15, N:3-14, D:4-13, M:5-12, E:6-11, P:7-10, L:8-9, S:16-17
Fa23 Q1b	SCCs	Kosaraju's; 9 SCCs (one per room if no cycles)
Fa23 Q2a	Dijkstra false	FALSE; fails on negative weights; counter-example shown
Fa23 Q2b	BFS O(V+E)	BFS for uniform-weight graphs
Fa23 Q2c (A)	MST islands	Blaze-Coral, Aqua-Blaze, Coral-Dune, Dune-Ember, Dune-Flare, Ember-Glimmer; Total=37
Fa23 Q2c (B)	MST islands	Coral-Flare, Blaze-Ember, Blaze-Dune, Blaze-Coral, Aqua-Blaze, Ember-Glimmer; Total=24
Fa23 Q3a	Frac. Knapsack	Set A: $147.00; Set B: $124.88

Q	Topic	Answer / Key Result
Fa23 Q3b (i)	Huffman codes	u=000, t=001, B=0100, l=0101, o=0110, h=0111, e=100, sp=101, ...
Fa23 Q3b (ii)	Decode	"Becheth el elaaerehel"
Fa23 Q4 (A)	DNA score	seq(5,5) = -1
Fa23 Q4 (B)	DNA score	seq(4,6) = -1
Fa23 Q4c	Time complexity	O(N x M)
Fa23 Q4d	Space optim.	O(min(N,M)) using rolling 2-row array

— End of Complete Solutions —